

SigaUTS

Sistema Integrado de Gestión Académica UTS

Integrantes:

Karen Lizeth Arenas Medina

Martin Enrique Bernal Gómez

UNIDADES TECNOLÓGICAS DE SANTANDER

Facultad de Ciencias Naturales e Ingeniería

Documento Inicial del Proyecto – Programación en Java B191
Tecnología en Desarrollo de sistemas informáticos

01 de mayo de 2026

Tabla de Contenido

1. Objetivos del Proyecto	3
1.1 Objetivo General	3
1.2 Objetivos Específicos	3
2. Justificación.....	3
3. Entorno de Trabajo	3
3.1 Tecnologías Backend.....	3
3.2 Tecnologías Frontend	4
3.3 Herramientas de Desarrollo	4
4. Requerimientos del Sistema.....	5
4.1 Requerimientos Funcionales	5
4.2 Requerimientos No Funcionales.....	5
5. Historias de Usuario	7
6. Base de Datos	9
6.1 Descripción del Modelo	9
6.2 Script SQL – Creación de Base de Datos e Inserción de Datos	9
6.3 Diagrama Entidad-Relación (DER)	11
7. Repositorio GitHub.....	12

1. Objetivos del Proyecto

1.1 Objetivo General

Desarrollar un sistema web de gestión académica (SigaUTS) para las Unidades Tecnológicas de Santander, que permita administrar usuarios, materias, grupos y matrículas estudiantiles de manera organizada y eficiente utilizando Spring Boot.

1.2 Objetivos Específicos

- Gestionar usuarios y roles dentro del sistema.
- Permitir la matrícula de estudiantes en diferentes grupos.
- Organizar la información académica de estudiantes y docentes.
- Implementar una base de datos relacional para el control de la información.
- Garantizar seguridad y acceso adecuado a la plataforma.

2. Justificación

Las Unidades Tecnológicas de Santander (UTS) es una institución de educación superior que maneja un gran volumen de información académica semestre a semestre. La gestión manual o por sistemas desarticulados genera demoras, errores en el registro de notas y dificultades para el seguimiento del rendimiento estudiantil.

SigaUTS busca solucionar esta problemática centralizando en una sola plataforma web todos los procesos académicos clave: desde el registro de los actores (estudiantes y docentes) hasta la consulta de calificaciones. Esto mejora la eficiencia administrativa, garantiza la integridad de los datos y proporciona una experiencia de usuario clara para cada rol del sistema.

El proyecto también representa una oportunidad para aplicar los conocimientos adquiridos en programación orientada a objetos, bases de datos relacionales, arquitectura MVC y desarrollo web con tecnologías actuales del mercado como Spring Boot, Thymeleaf o React, y MySQL.

3. Entorno de Trabajo

3.1 Tecnologías Backend

- Lenguaje: Java 17
- Framework: Spring Boot 3.x

- Seguridad: Spring Security
- Persistencia: Spring Data JPA / Hibernate
- Base de Datos: MySQL 8
- Construcción: Maven

3.2 Tecnologías Frontend

- Motor de plantillas: Thymeleaf o React (por definir)
- Estilos: Bootstrap 5
- Herramienta de diseño: Figma (prototipos)

3.3 Herramientas de Desarrollo

- IDE: IntelliJ IDEA / VS Code
- Control de versiones: Git + GitHub
- Gestión del proyecto: GitHub Projects (tablero Kanban)
- Documentación de API: Swagger / OpenAPI
- Pruebas: JUnit 5 + Mockito
- Entorno local: Docker (opcional) o instalación directa

4. Requerimientos del Sistema

4.1 Requerimientos Funcionales

ID	Nombre	Descripción
RF-01	Registro de estudiantes	El sistema debe permitir registrar, editar y eliminar información de estudiantes (nombre, código, programa, semestre).
RF-02	Registro de docentes	El sistema debe permitir registrar, editar y eliminar información de docentes (nombre, cédula, especialidad, correo).
RF-03	Gestión de materias	El sistema debe permitir crear, editar y eliminar materias con sus respectivos créditos y descripción.
RF-04	Asignación de materias a docentes	El sistema debe permitir asignar una o más materias a un docente por período académico.
RF-05	Gestión de horarios	El sistema debe permitir crear y visualizar horarios de clases por grupo, aula y docente.
RF-06	Matrícula de estudiantes	El sistema debe permitir matricular estudiantes en materias para un semestre específico.
RF-07	Registro de calificaciones	El sistema debe permitir a los docentes ingresar y editar las notas de los estudiantes por corte.
RF-08	Consulta de notas	Los estudiantes deben poder consultar sus calificaciones por materia y período.
RF-09	Generación de reporte académico	El sistema debe generar un reporte del rendimiento académico por estudiante, materia o programa.
RF-10	Gestión de usuarios y roles	El sistema debe soportar tres roles: Administrador, Docente y Estudiante, con permisos diferenciados.

4.2 Requerimientos No Funcionales

ID	Atributo	Descripción
RNF-01	Rendimiento	El sistema debe responder en menos de 3 segundos ante cualquier consulta con carga normal.

RNF-02	Seguridad	Los datos de usuarios deben estar protegidos con autenticación JWT y contraseñas cifradas con BCrypt.
RNF-03	Usabilidad	La interfaz debe ser intuitiva, siguiendo principios de UX para usuarios con conocimientos básicos de computación.
RNF-04	Disponibilidad	El sistema debe estar disponible el 95% del tiempo en horario académico (6am - 10pm).
RNF-05	Escalabilidad	La arquitectura debe permitir añadir nuevos módulos sin afectar los existentes.
RNF-06	Compatibilidad	El sistema debe funcionar en navegadores modernos: Chrome, Firefox y Edge (últimas 2 versiones).
RNF-07	Mantenibilidad	El código debe seguir buenas prácticas (clean code), con documentación Javadoc en clases principales.
RNF-08	Portabilidad	El sistema debe poder desplegarse en sistemas Windows y Linux con Java 17+ y MySQL 8+.

5. Historias de Usuario

Las historias de usuario describen las funcionalidades del sistema desde la perspectiva de cada tipo de usuario, priorizando el valor entregado y los criterios de aceptación que validan su correcta implementación.

HU-01 – Registro de nuevo estudiante	
Como un(a):	Administrador
Quiero:	registrar un nuevo estudiante en el sistema con todos sus datos personales y académicos
Para:	que quede inscrito en la plataforma y pueda acceder a sus servicios académicos.
Criterios de aceptación:	1. El formulario debe validar que el código de estudiante sea único. 2. Todos los campos obligatorios (nombre, código, programa, semestre) deben ser diligenciados. 3. Tras el registro exitoso, el sistema envía confirmación y crea el usuario con rol Estudiante.

HU-02 – Ingreso de calificaciones	
Como un(a):	Docente
Quiero:	ingresar las calificaciones de mis estudiantes por corte en cada materia que dicto
Para:	llevar un control formal del rendimiento académico y que los estudiantes puedan consultarlas.
Criterios de aceptación:	4. Solo el docente asignado a la materia puede ingresar notas. 5. El sistema valida que las notas estén en el rango de 0.0 a 5.0. 6. Una vez guardadas, las notas son visibles para el estudiante en su perfil.

HU-03 – Consulta de notas	
Como un(a):	Estudiante
Quiero:	consultar mis calificaciones por materia y por corte en el semestre actual
Para:	conocer mi rendimiento académico y tomar decisiones oportunas.

Criterios de aceptación:	7. El estudiante solo puede ver sus propias calificaciones. 8. Se muestran las notas por corte y el promedio calculado automáticamente. 9. Si aún no hay notas registradas, el sistema muestra un mensaje informativo.
---------------------------------	--

HU-04 – Gestión de horarios

Como un(a):	Administrador
Quiero:	crear y asignar horarios de clases a grupos, aulas y docentes
Para:	organizar la distribución académica del semestre sin conflictos de tiempo o espacio.
Criterios de aceptación:	10. El sistema alerta si hay cruce de horario para un docente o aula. 11. Cada horario debe tener: materia, docente, aula, día y hora. 12. Los horarios publicados son visibles para estudiantes y docentes.

HU-05 – Matrícula de materias

Como un(a):	Estudiante
Quiero:	matricularme en las materias disponibles para el siguiente semestre
Para:	formalizar mi inscripción académica y quedar asignado a los grupos correspondientes.
Criterios de aceptación:	13. El estudiante solo puede matricularse en materias habilitadas para su programa y semestre. 14. El sistema valida que no se exceda el límite de créditos permitidos. 15. Al confirmar la matrícula, el estudiante recibe un resumen de las materias inscritas.

6. Base de Datos

6.1 Descripción del Modelo

El modelo relacional de SigaUTS está compuesto por las siguientes entidades principales: Usuario, Estudiante, Docente, Programa, Materia, Grupo, Horario, Matricula y Calificacion. Las relaciones entre estas entidades garantizan la integridad referencial del sistema y soportan todos los requerimientos funcionales definidos.

6.2 Script SQL – Creación de Base de Datos e Inserción de Datos

```
=====
-- SIGAUTS - Script de Base de Datos
```

```
-- Motor: MySQL 8  Charset: utf8mb4
```

```
-- =====
```

```
CREATE DATABASE IF NOT EXISTS sigauts CHARACTER SET utf8mb4 COLLATE
utf8mb4_spanish_ci;
USE sigauts;
```

```
-- Tabla: usuario (base para login y roles)
```

```
CREATE TABLE usuario (
  id_usuario INT PRIMARY KEY AUTO_INCREMENT,
  nombre VARCHAR(100) NOT NULL,
  correo VARCHAR(100) NOT NULL UNIQUE,
  contraseña VARCHAR(255) NOT NULL,
  rol ENUM('ADMINISTRADOR','DOCENTE','ESTUDIANTE') NOT NULL,
  activo TINYINT(1) DEFAULT 1
);
```

```
-- Tabla: estudiante
```

```
CREATE TABLE estudiante (
  id_estudiante INT PRIMARY KEY AUTO_INCREMENT,
  id_usuario INT NOT NULL UNIQUE,
  codigo VARCHAR(15) NOT NULL UNIQUE,
  semestre TINYINT UNSIGNED NOT NULL,
  id_programa INT NOT NULL,
  FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
);
```

```
-- Tabla: docente
```

```
CREATE TABLE docente (
  id_docente INT PRIMARY KEY AUTO_INCREMENT,
  id_usuario INT NOT NULL UNIQUE,
  cedula VARCHAR(15) NOT NULL UNIQUE,
  especialidad VARCHAR(100),
```

```
FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
);

-- Tabla: materia
CREATE TABLE materia (
    id_materia INT PRIMARY KEY AUTO_INCREMENT,
    nombre VARCHAR(100) NOT NULL,
    codigo VARCHAR(10) NOT NULL UNIQUE,
    creditos TINYINT UNSIGNED NOT NULL,
    id_programa INT NOT NULL,
);

-- Tabla: grupo (materia + docente + semestre + periodo)
CREATE TABLE grupo (
    id_grupo INT PRIMARY KEY AUTO_INCREMENT,
    id_materia INT NOT NULL,
    id_docente INT NOT NULL,
    periodo VARCHAR(10) NOT NULL,
    nombre_grupo VARCHAR(50),
    cupo_maximo INT
    FOREIGN KEY (id_materia) REFERENCES materia(id_materia),
    FOREIGN KEY (id_docente) REFERENCES docente(id_docente)
);

-- Tabla: matricula (estudiante inscrito en un grupo)
CREATE TABLE matricula (
    id_matricula INT PRIMARY KEY AUTO_INCREMENT,
    id_estudiante INT NOT NULL,
    id_grupo INT NOT NULL,
    fecha_matricula DATE NOT NULL,
    estado ENUM('Activa','Cancelada','Finalizada') DEFAULT 'Activa',
    UNIQUE KEY uk_matricula (id_estudiante, id_grupo),
    FOREIGN KEY (id_estudiante) REFERENCES estudiante(id_estudiante),
    FOREIGN KEY (id_grupo) REFERENCES grupo(id_grupo)
);

-- =====
-- INSERCIÓN DE DATOS DE PRUEBA
-- =====

-- Usuarios (contraseñas hasheadas con BCrypt en producción)
INSERT INTO usuario (nombre, correo, contrasena, rol) VALUES
    ('Admin Sistema', 'admin@uts.edu.co', '$ADMIN', 'ADMINISTRADOR'),
    ('Carlos Pérez', 'cperez@uts.edu.co', '$DOC1', 'DOCENTE'),
    ('María López', 'mlopez@uts.edu.co', '$DOC2', 'DOCENTE'),
    ('Juan García', '2023001@uts.edu.co', '$EST1', 'ESTUDIANTE'),
```

```

('Ana Torres', '2023002@uts.edu.co', '$EST2', 'ESTUDIANTE'),
('Luis Martínez', '2022015@uts.edu.co', '$EST3', 'ESTUDIANTE');
    
```

-- Docentes

```

INSERT INTO docente (id_usuario, cedula, especialidad) VALUES
(2, '12345678', 'Bases de Datos y Programación Java'),
(3, '87654321', 'Redes y Sistemas Operativos');
    
```

-- Estudiantes

```

INSERT INTO estudiante (id_usuario, codigo, semestre) VALUES
(4, '2023001', 3),
(5, '2023002', 3),
(6, '2022015', 5);
    
```

-- Materias

```

INSERT INTO materia (nombre, codigo, creditos) VALUES
('Programación Orientada a Objetos', 'POO201', 4),
('Bases de Datos I', 'BD201', 4),
('Estructuras de Datos', 'ED301', 3);
    
```

-- Grupos

```

INSERT INTO grupo (id_materia, id_docente, periodo) VALUES
(1, 1, '2026-1'),
(2, 1, '2026-1'),
(3, 2, '2026-1');
    
```

-- Matrículas

```

INSERT INTO matricula (id_estudiante, id_grupo, fecha_matricula) VALUES
(1, 1, '2026-02-15'),
(1, 2, '2026-02-15'),
(2, 1, '2026-02-16'),
(2, 2, '2026-02-17'),
(3, 3, '2026-02-18');
    
```

6.3 Diagrama Entidad-Relación (DER)

A continuación, se describe la estructura del DER del sistema SigaUTS. El diagrama completo en formato imagen se adjunta en el repositorio GitHub del proyecto (ver carpeta /docs/DER_SigaUTS.png).

Entidad	Atributos Principales	Relaciones
programa	id_programa, nombre, codigo, modalidad	1:N con estudiante, 1:N con materia

usuario	id_usuario, nombre, correo, contraseña, rol	1:1 con estudiante / docente
estudiante	id_estudiante, codigo, semestre	N:M con grupo (vía matricula)
docente	id_docente, cedula, especialidad	1:N con grupo
materia	id_materia, nombre, codigo, creditos	1:N con grupo
grupo	id_grupo, periodo, numero_grupo	1:N con horario, 1:N con matricula
horario	id_horario, dia, hora_inicio, hora_fin, aula	N:1 con grupo
matricula	id_matricula, fecha_matricula, estado	1:N con calificacion
calificacion	id_calificacion, corte, nota, fecha_registro	N:1 con matricula

7. Repositorio GitHub

El repositorio del proyecto se encuentra alojado en GitHub bajo la organización del grupo de trabajo. La estructura propuesta es la siguiente:

- README.md – Descripción general del proyecto
- /docs – Documento inicial, DER e imágenes de referencia
- /backend – Proyecto Spring Boot (src, pom.xml, application.properties)
- /database – Script SQL de creación e inserción de datos
- /frontend – Plantillas Thymeleaf o proyecto React (según decisión final)